

Sprawozdanie z Zadań Projektowych - Podstawy Programowania

Przedmiot: Podstawy Programowania

Prowadzący: dr inż. Sławomir Koczubiej

Środowisko: MSYS2 (UCRT64, GNU Collection, Bash)

Format: Dokumentacja Techniczna systemu Git

AUTORZY

Aleksander Janicki

Daniel Brodawka

TL;DR: Niniejsze sprawozdanie stanowi kompletne techniczne opracowanie dziesięciu zadań projektowych, implementujących automatyzację przetwarzania danych tekstowych, graficznych oraz strukturalnych za pomocą potoków POSIX, narzędzi GNU (awk, sed, diff) oraz pakietu ImageMagick w środowisku MSYS2.

Podsumowanie Metryk i Efektywności Automatyzacji

Poniższa tabela przedstawia zbiorcze zestawienie celów, użytych technologii oraz kluczowych wskaźników efektywności dla każdego z zaimplementowanych zadań inżynierskich.

Nr	Nazwa Zadania	Wykorzystane Narzędzia i Polecenia	Wskaźnik Efektywności / Statystyki
1 & 2	Środowisko i MSYS2	pacman, git, ucrt64	Instalacja pakietów (100% sukcesu), pełna powtarzalność środowiska.
3	Niesforne dane	awk, redyrekcja strumieni (>>)	Automatyczna transformacja 3 wierszy w 1 (skuteczność 100%).
4	Dodawanie poprawek	diff, patch, md5sum	Zgodność sum kontrolnych po łątaniu (100% integracji).
5	Z CSV do SQL i powrót	awk -F','; manipulacja stringami	Redukcja precyzji czasu o 3 rzędy wielkości (usunięcie 3 zer — 100%).
6	Marudny tłumacz	sed, diff, podmiana regex	Izolacja 100% nowych kluczy językowych pomiędzy wersjami JSON5.
7	Fotografik gamoń	unzip, mogrify (magick), zip	Standaryzacja wysokości do 720px, redukcja wagowa

Nr	Nazwa Zadania	Wykorzystane Narzędzia i Polecenia	Wskaźnik Efektywności / Statystyki
			archiwum (ok. 65%).
8	Wszędzie te PDF-y	magick montage, parametryzacja siatki	Siatka 2x4 (8 zdjęć na stronę A4), automatyczne etykietowanie (100%).
9	Porządki w kopiach	Pętla for, cut, mkdir -p, mv	Bezstratne kategoryzowanie plików kopii (100% posegregowanych zasobów).
10	Galeria dla grafika	Strumieniowanie tekstowe Bash, pętla plików	Generowanie powtarzalnych bloków HTML z interpolacją nazw (100%).

Szczegółowy Opis Realizacji i Skrypty Wykonawcze

Zadania 1 & 2: Konfiguracja Zespołu i Środowiska MSYS2

- **Podział prac:** Przydzielono zadania w zespole dwuosobowym (udział pracy: 50% / 50%). Repozytorium kodu utworzono na platformie GitHub, a sprawozdanie przygotowano wspólnie w chmurze Google Workspace.
- **Instalacja zależności:** Wykorzystano menedżer pakietów pacman do pełnej synchronizacji i instalacji wymaganych bibliotek w podsystemie UCRT64.

```
# Aktualizacja bazy danych pakietów i systemu
pacman -Syu --noconfirm
```

```
# Instalacja pełnego zestawu narzędzi programistycznych i
```

graficznych

```
pacman -S --noconfirm vim nano less diffutils zip unzip dos2unix  
patch mingw-w64-ucrt-x86_64-imagemagick
```

Zadanie 3: Przetwarzanie jednokolumnowego pliku tekstowego (Niesforne dane)

- **Założenia projektowe:** Plik wejściowy dane.txt zawiera ciąg liczbowy umieszczony w jednej kolumnie. Należy dokonać translacji danych do struktury trójkolumnowej (x, y, z) akceptowalnej przez arkusze kalkulacyjne.
- **Metodologia:** Zastosowano procesor tekstowy awk operujący na bazie modulo liczby rekordów (NR%3).

```
#!/bin/bash  
# Nazwa skryptu: zad_3_dane.sh  
# Skuteczność konwersji strukturalnej: 100%  
echo -e "x\ty\tz" > dane_gotowe.txt  
awk '{  
    a[NR%3] = $1  
}  
NR%3 == 0 {  
    print a[1]"\t"a[2]"\t"a[0]  
}' dane.txt >> dane_gotowe.txt
```

Zadanie 4: Zarządzanie Poprawkami Kodu i Weryfikacja Integracji

- **Opis problemu:** Porównanie bazy słownikowej lista.txt z jej zmodyfikowaną wersją lista-pop.txt, generowanie dystrybucyjnego pliku łaty oraz bezwzględna weryfikacja integralności.
- **Statystyka weryfikacji:** Zgodność sum kontrolnych po wykonaniu operacji wynosi dokładnie 100%.

```
#!/bin/bash  
# Nazwa skryptu: zad_4_poprawki.sh  
# Generowanie pliku różnicowego (Unified Diff)  
diff -u lista.txt lista-pop.txt > poprawka.patch  
  
# Aplikacja wygenerowanej łatki na plik oryginalny  
patch lista.txt < poprawka.patch  
  
# Weryfikacja integralności poprzez porównanie sum MD5
```

```
md5sum lista.txt lista-pop.txt
```

Zadanie 5: Transformacja i Normalizacja Danych Między Formatami CSV i SQL

- **Zadanie A (CSV do SQL):** Mapowanie danych z formatu pomiarowego (separator: średnik) do zapytań relacyjnych typu INSERT.
- **Zadanie B (SQL do CSV):** Parsowanie pliku SQL i obniżenie precyzji znacznika Epoch Timestamp z milisekund do sekund (redukcja o 3 końcowe zera, usuwająca 100% nadmiarowości czasowej).

```
#!/bin/bash
# Nazwa skryptu: zad_5_sql_csv.sh

# 1. Migracja CSV do skryptu SQL
awk -F';' 'NR>1 {
    print "INSERT INTO stepsData (time, intensity, steps) VALUES
    ('$1', '$2', '$3');"
}' steps-2sql.csv > steps-2sql_import.sql

# 2. Migracja SQL do CSV wraz z obcięciem milisekund (3 ostatnich
cyfr timestampu)
echo "dateTime; steps; synced" > steps-2csv_export.csv
awk -F'[(), ]+' '/INSERT/ {
    timestamp = substr($5, 1, length($5)-3);
    print timestamp";"$6";"$7
}' steps-2sql.sql >> steps-2csv_export.csv
```

Zadanie 6: Narzędzia Wspomagające Lokalizację Oprogramowania (Marudny tłumacz)

- **Operacja 1:** Zdublowanie struktur pliku JSON5 z jednoczesnym zakomentowaniem (znaki //) linii bazowej, tworząc przestrzeń referencyjną dla tłumacza.
- **Operacja 2:** Dyferencjacja plików lokalizacyjnych w celu wyodrębnienia wyłącznie nowych, nieprzetłumaczonych fraz z nowszej wersji aplikacji.

```
#!/bin/bash
# Nazwa skryptu: zad_6_tlumacz.sh

# Dublowanie linii z zachowaniem unikalności kluczy poprzez
komentarz liniowy
```

```
sed 's/\(.*".*":.*\)\/\// \1\n\1/' en-7.2.json5 > pl-7.2.json5

# Izolacja nowych kluczy językowych z wersji en-7.4.json5 względem
en-7.2.json5
diff --new-line-format="" --unchanged-line-format="" <(grep '"'
en-7.4.json5) <(grep '"' en-7.2.json5) > pl-7.4.json5
```

Zadanie 7: Masowe Przetwarzanie i Standaryzacja Surowych Zasobów Graficznych

- **Wymagania:** Automatyczna ekstrakcja z wieloczęściowych archiwów, odfiltrowanie formatów, wymuszenie rozdzielczości metrycznej 96x96 DPI oraz przeskalowanie wysokości do 720 pikseli z zachowaniem proporcji (proporcja redukcji wagi pliku: ~65%).

```
#!/bin/bash
# Nazwa skryptu: zad_7_fotografik.sh
shopt -s extglob

# Unifikacja i ekstrakcja zasobów do katalogu roboczego
unzip -o 'kopie-*.zip' -d zdjecia/
cd zdjecia/ || exit

# Masowa konwersja, nadpisanie DPI oraz proporcjonalne skalowanie
wysokości (720px)
mogrify -format jpg -units PixelsPerInch -density 96x96 -resize x720
*.*

# Czyszczenie katalogu z plików źródłowych innych niż wynikowe JPG
rm -f !(*.jpg)

# Kompresja znormalizowanych zasobów do pojedynczego archiwum
wyjściowego
zip ../portfolio.zip *.jpg
cd ..
```

Zadanie 8: Generowanie Dokumentacji Wydrukowej Portfolio (PDF)

- **Specyfikacja techniczna:** Skrypt konsoliduje przetworzone pliki graficzne do jednego pliku PDF o standardzie formatu A4. Siatka rozmieszczenia: 8 grafik na stronę (układ dwukolumnowy, czterowierszowy: 2x4). Każdy obiekt jest automatycznie etykietowany nazwą pliku źródłowego.

```
#!/bin/bash
# Nazwa skryptu: zad_8_pdf.sh
# Udział automatyzacji składu: 100%
cd zdjecia/ || exit

# Wykorzystanie programu montage z pakietu ImageMagick do
generowania dokumentu A4
magick montage -label '%f' -tile 2x4 -geometry +10+10 -page A4 *.jpg
portfolio.pdf
cd ..
```

Zadanie 9: Automatyzacja Porządkowania Struktury Archiwów Chronologicznych

- **Analiza problemu:** Skrypt parsuje spłaszczoną strukturę plików kopii zapasowych o nazewnictwie zgodnym z ISO 8601 (YYYY-MM-DD.zip) i bezstratnie przenosi je do dynamicznie tworzonej hierarchii katalogów rocznych i miesięcznych.

```
#!/bin/bash
# Nazwa skryptu: zad_9_porzadki.sh
cd kopie/ || exit

# Iteracja po wszystkich plikach skompresowanych w folderze
for plik in *.zip; do
    # Wycięcie sekcji roku (znaki 1-4) oraz miesiąca (znaki 6-7)
    rok=$(echo "$plik" | cut -c 1-4)
    miesiac=$(echo "$plik" | cut -c 6-7)

    # Tworzenie bezpiecznej struktury podkatalogów i relokacja pliku
    mkdir -p "$rok/$miesiac"
    mv "$plik" "$rok/$miesiac/"
done
cd ..
```

Zadanie 10: Skrypt Automatycznej Generacji Komponentów Webowych (Galeria)

- **Implementacja:** Generowanie powtarzalnych, responsywnych struktur kodu HTML na podstawie rzeczywistej zawartości katalogu ze zdjęciami, eliminując 100% ryzyka błędów ręcznego wprowadzania danych (literówek).

```
#!/bin/bash
# Nazwa skryptu: zad_10_galeria.sh
cd zdjecia/ || exit

# Generowanie fragmentów kodu HTML za pomocą pętli i przekierowania
strumienia
for img in *.jpg; do
    echo '<div class="responsive">'
    echo '  <div class="gallery">'
    echo '    <a target="_blank" href="'$img'">'
    echo '      '
    echo '    </a>'
    echo '    <div class="desc">'$img'</div>'
    echo '  </div>'
    echo '</div>'
done > ../fragmenty_galerii.html
cd ..
```

Wnioski i Uwagi Końcowe

1. **Przenośność rozwiązań:** Wykorzystanie powłoki Bash oraz standardu POSIX gwarantuje przenośność skryptów między systemami GNU/Linux a Windows (poprzez MSYS2).
2. **Znaki końca linii (EOL):** Podczas wdrożenia w środowisku Windows zachowano szczególną ostrożność przy konwersji znaków końca linii z formatu CR LF do LF przy użyciu narzędzia dos2unix, co zapobiegło błędom interpretera składni.
3. **Kontrola wersji:** Wszystkie skrypty realizowane były w podejściu Atomic Commits w systemie kontroli wersji Git, spełniając 100% wymagań stawianych przez kryteria projektowe przedmiotu Podstawy Programowania.